

Information Borrowing

Hazel Torek

Clemson University
Clemson, South Carolina, USA
ctorek@clemson.edu

Hemant Gouni

Carnegie Mellon University
Pittsburgh, Pennsylvania, USA
hsgouni@cs.cmu.edu

Jonathan Aldrich

Carnegie Mellon University
Pittsburgh, Pennsylvania, USA
jonathan.aldrich@cs.cmu.edu

Abstract

In Rust and other languages with ownership type systems, *borrowing* allows creating references that are eventually returned to their owners. We motivate and introduce *information borrowing*, which enables temporary access to information rather than temporary access to memory. This enables us to reason about whether information escapes a given scope. We demonstrate its usage towards establishing security boundaries around untrusted library code, then elaborate it via recent work on information flow typing.

1 Introduction

When speaking about the security properties of a program, reasoning about *non-escape* plays an essential role. For instance, a program which checks passwords may wish to ensure that a user’s password isn’t written to insecure locations or sent over the network. Consider the benevolent and malevolent variants of a function that check various password strength requirements in Figure 1, where the malevolent implementation transmits the password over the network. Although the behavior is clear when looking at the implementation, these two functions are impossible to distinguish using their types; identifying such issues scales poorly with untrusted third-party library code of arbitrary complexity.

To address the preceding concerns, we introduce the notion of *information borrowing* to expose them at the type level. Similarly to borrowing in languages like Rust [4, 12], where memory must be returned in its entirety to its caller, borrowed information must be returned in its entirety to the caller. This means that borrowed information must be revealed only through the return type. In so doing, we will be able to specify in the type signature of a function whether information passed to it is allowed to escape.

We start with a tour of the programming features we introduce in Section 2. Their elaboration into the core theory, based on prior work in information flow [3], follows in Section 3. Section 4 outlines the treatment for realistic features including mutable state. Section 5 compares to related work.

2 Approach

We first import a simple treatment of effects: programs which perform I/O will be marked as such in their type. We then extend the syntax to include the type $\&T$ for any type T , which

```
1 val len : string -> int      9 val send : string -> unit
2 val hasSpecialChar :        10
3   string -> bool           11 val valid' :
4                               12   string -> bool
5 val valid : string -> bool   13 let valid' = fun pw ->
6 let valid = fun pw ->       14   send(pw);
7   len(pw) >= 10 &&          15   ret len(pw) >= 10 &&
8   hasSpecialChar(pw)        16   hasSpecialChar(pw)
```

Figure 1. Benevolent and malevolent examples of a program that checks whether a user’s password is secure.

means that the parameter of type T is being *information* borrowed, and the dereference syntax $*t$, which eliminates a borrowed term t of type $\&T$ into the underlying value of type T . Using this syntax, we can rewrite the examples from Figure 1 to safely borrow the password string and demonstrate that it will not be leaked by the function’s implementation.

The `valid` function calls two auxiliary functions; we may suspect that these also need to respect the information borrow. There are two options for proceeding:

- Dereference the information-borrowed variable, passing $*pw$ to each of the auxiliary functions, and ensure that the return types of auxiliary functions don’t represent possible leakage of information.
- Have both functions borrow their parameters, taking responsibility for their confinement.

2.1 Pass By Dereference

The first approach is to turn an information-borrowed variable back into a normal one, incurring restrictions to ensure the original borrow is still respected. In Figure 2, the functions `len` and `hasSpecialChar` take parameters of type `string`, but the variable `pw` is of type `&string` since it is information-borrowed. Accordingly, we *dereference* `pw` to obtain a term of type `string` that can be passed to each. For

```
1 val len : string -> int      9 val send :
2 val hasSpecialChar :        10   string -> [io] unit
3   string -> bool           11
4                               12 val valid' :
5 val valid : &string -> bool  13   &string -> [io] bool
6 let valid = fun pw ->       14 let valid' = fun pw ->
7   len(*pw) >= 10 &&          15   send(*pw); △
8   hasSpecialChar(*pw)      16   len(*pw) >= 10 &&
                               17   hasSpecialChar(*pw)
```

Figure 2. Information-borrowed validator example that uses dereferences to call auxiliary functions.

```

1 val attempt : &string
2
3 val check : string -> [io] bool
4 let check = fun s ->
5   io.log("login attempt");
6   *s == "pass123"
7
8 let _ : [io] bool = check *attempt  $\Delta$ 

```

Figure 3. Example showing the lack of granularity required when dereferencing information-borrowed variables.

```

1 val len : &string -> int
2 val hasSpecialChar :
3   &string -> bool
4
5 val valid : &string -> bool
6 let valid = fun pw ->
7   len(pw) >= 10 &&
8   hasSpecialChar(pw)
9
10 val send :
11   string -> [io] unit
12
13 val valid' :
14   &string -> bool
15
16 let valid' = fun pw ->
17   send(pw);  $\Delta$ 
18   len(pw) >= 10 &&
19   hasSpecialChar(pw)

```

Figure 4. Information-borrowed validator example that forces auxiliary functions to borrow their parameters.

the example on the right in Figure 2, the dereferenced password is passed to send, which contains *io* in its return type. Accordingly, the dereference operation fails: the marker for the *io* effect reveals the passed information might escape.

Specifically, locations where a dereference occurs must not invoke effects capable of leaking the data. Notably, this doesn't distinguish whether the effect in question *does* leak the data, just whether it would be *capable* of doing so. For example, consider Figure 3; the program here checks whether a user's password attempt is correct. However, calling the check function with a borrowed variable using dereferencing fails, because the presence of the *io* effect means that attempt must be assumed to be leaked, despite the body of check not doing so. We resolve this using the second approach previously mentioned.

2.2 Pass By Borrow

Our other option is to require auxiliary function calls to also borrow their parameters. Figure 4 modifies the signatures of len and hasSpecialChar accordingly. By doing so, we can ensure that these functions will themselves respect the rules of the information borrow; we need not check their signatures for non-leakage and can simply pass the borrowed parameter pw directly to them. In this case, the error in the malevolent example arises because send does not borrow its parameter and pw is passed without being dereferenced, resulting in a simple type mismatch. Even if we were to dereference pw before passing it, this would still fail for the same reason as in Figure 2 due to the presence of effects.

This approach provides finer granularity than the first when tracking the flow of borrowed variables. Whereas the first approach doesn't track *which* information is flowing to

```

1 val attempt : &string
2
3 val check : &string -> [io] bool
4 let check = fun s ->
5   io.log("login attempt");
6   *s == "pass123"
7
8 let _ : [io] bool = check attempt

```

Figure 5. Example showing the finer granularity obtained when using information borrowing in auxiliary functions.

an untrusted sink, this approach can keep track of whether borrowed information is being leaked or the effect is irrelevant. For example, consider Figure 5; in this example, the final call to check succeeds because the parameter to check is borrowed, guaranteeing that it won't leak since the *io* effect is independent of the borrowed variable attempt.

However, this approach might not always be plausible when working with third-party libraries, since it might not be possible to modify the type signatures or behavior of external functions, meaning that while it allows for extra precision, it is slightly less general than the first.

3 Elaboration

Information borrowing is enabled by three key constructs of the underlying language, a calculus for information flow based on that of Gouni et al. [3]. These are (1) the *quasi-closed modality*, which represents information-borrowed variables, (2) the *quasi-open modality*, which represents effects, and (3) *quantifiers*, which are used to introduce new dependency variables. The last is essential; we specifically build on the work of Gouni et al. [3] because other theories of information flow lack the necessary treatment of quantifiers to implement information borrowing. To demonstrate how all these construct are used in concert, we show at the end of the section the elaboration of the malevolent password validator example from Figure 2 into the core calculus in Figure 8.

The typing judgment $\Gamma; \phi \vdash e : A$ can be read “under the variables $x : A$ in Γ and ambient dependencies ϕ , e has type A .” We proceed by reviewing in turn each construct above.

3.1 Quasi-Open Modality

We show the quasi-open modality in Figure 6. It is introduced by the consume(–) operation, specifically by capturing certain ambient dependencies ϕ within the modality. Its eliminator is produce(–), which releases dependencies captured by

$\frac{\Gamma; \phi' \vdash e : A}{\Gamma; \phi' \vdash \text{consume}(e) : \odot_{\phi}(A)}$	$\frac{\Gamma; \phi' \vdash e : \odot_{\phi}(A)}{\Gamma; \phi \vdash \text{produce}(e) : A}$
---	--

Figure 6. Typing rules for the open modality, used to represent effects.

$$\begin{array}{c}
\text{T-SEAL} \\
\frac{\Gamma; \phi \vdash e : A}{\Gamma; \phi \vdash \text{seal}[\phi'](e) : \bullet_{\phi'}(A)} \\
\\
\text{T-UNSEAL} \\
\frac{\Gamma; \phi \vdash e : \bullet_{\phi'}(A) \quad \Gamma, x : A; \phi \vdash e_1 : B \quad \phi \vdash B \geq \phi'}{\Gamma; \phi \vdash \text{unseal } e \text{ in } x.e_1 : B}
\end{array}$$

Figure 7. Typing rules for the quasi-closed modality, used to ensure the safety of borrowed variables.

the modality back into the ambient dependencies. Variants of it have appeared in prior work on information flow [5, 10].

Importantly, the open modality is a *negative type*, meaning $\text{consume}(-)$ suspends its contents—it contains a computation. As a result, rather than using a separate effect system for tracking side-effecting operations, we can use dependencies situated within the open modality to do so instead. The intuition is that we can think of the ambient dependencies ϕ as, in reality, the ambient *effects*. For instance, the `send` operation from Figure 2 induces a dependency on `io`; indeed, `[io] unit` translates to $\bigcirc_{\text{io}}(\text{unit})$ within the core calculus. So long as the execution of the enclosing program continues to involve `send`, the `io` dependency induced by it will be ‘stuck’ to the program. This flags it as potentially leaking data passed to it, to be exploited by the next construct.

Note that in the preceding examples, we did not have explicit syntax corresponding to $\text{consume}(-)$ and $\text{produce}(-)$, instead performing these operations implicitly on function introduction and application by piggybacking on the suspension semantics of functions. We now look to the quasi-closed modality, which uses the effect tracking provided by the open modality to implement information borrowing.

3.2 Quasi-Closed Modality

We show the quasi-closed modality in Figure 7; it is a slight variation upon the protection connective from the Dependency Core Calculus [1]. It is introduced by `seal[-](-)`, which wraps an expression in the modality. The rule T-UNSEAL acts as the eliminator, unwrapping the modality from its first argument and passing the contents to its second. Of critical importance is its final premise, which checks whether data protected under the modality has leaked. We cannot treat it in full here, but at a high level it fulfills two related functions.

First, it places the information in B appropriately under the quasi-closed modality. So if B contains a `bool`, then it must be appear inside $\bullet_{\phi'}(-)$; but a `unit` need not, since it represents no information. Second, it checks whether the contents of ϕ , representing the effects done in the body e_1 , betray having leaked information inappropriately.¹ To explain

¹This is in fact the main way in which our work extends Gouni et al. [3]

```

1 val valid : ∀α . (●[α](string) -> ●[α](bool))
2 let valid = fun pw ->
3   unseal pw in x.
4   send(x); Δ
5   seal(
6     len(x) >= 10 &&
7     hasSpecialChar(x)
8   )

```

Figure 8. Elaboration of the malevolent password validator example with dereferencing into the IFC language.

how, we must briefly digress to reveal the precise mechanics of dependencies in our framework. Each dependency, like `io`, is associated with a type conveying the information it represents when used to track a side effect. For instance, `io` is associated with type `string`, because this is the shape of data which gets written to I/O by operations like `send`. Other dependencies may have the `unit` type, representing no information and therefore behaving as a bog-standard information flow dependency, or more sophisticated types like $\bullet_{\phi}(\text{bool})$. The justification for these typed dependencies is that their types represent extraneous information besides that conveyed by the types within which they directly occur—that is, the information revealed by side effects. Intuitively, the `io` in `[io] unit` indicates that this type conveys more than just a `unit` worth of information. So with typed dependencies in hand, the final premise of T-UNSEAL may check whether leakage occurs by performing for the type of each dependency in ϕ the same check it performs for B : that all the information in the type is placed under the quasi-closed modality $\bullet_{\phi'}(-)$.

An information borrowed type is roughly represented within the core calculus as the quasi-closed modality; in particular, this modality is the fundamental construct allowing reasoning about non-escape. The dereference syntax `*` can be thought of as syntax sugar for unsealing. There is a final piece left before we can work a full example explicating our prior examples within the core calculus.

3.3 Quantification

Our framework supports universal quantifiers over dependencies. We introduce them by example in Figure 8, which elaborates the right side of Figure 2. We use a universal quantifier to bind a new dependency α within the type of `valid`. We place the argument and return types under the quasi-closed modality at α . This schema of usage for the just-discussed constructs provides information borrowing: information about the argument cannot escape outside of the function, because it is sealed under the quasi-closed modality at a newly-introduced α which does not exist outside the function’s scope. In particular, the role played by the quantifier is to *scope* the information borrow, because types occurring outside the quantifier cannot possibly place themselves under the appropriate modality at the α it introduces.

As such, no locations outside of the function may receive the information in its argument. The return value is the only vector through which this information might escape the function, being placed under the appropriate modality. In a function with borrowed parameters that calls functions which also use information borrowing, such as `hasSpecialChar` in Figure 4, the quantifier at the call site is instantiated with α . This ensures that the borrowed parameter can be passed to it, with its return value placed under the quasi-closed modality as needed. The information borrow can be ended by instantiating α itself to the empty set $\emptyset$ when calling `valid`, which has the effect of getting rid of the quasi-closed modality.

Looking to the body of `valid`, we model `*pw` in Figure 2 by first unsealing the function argument, binding it as `x`. We then invoke `send` on `x`, and return the result of the password validity check after sealing it to satisfy the quasi-closed modality as the return type. The reason that Figure 8 fails, then, is because calling `send(x)` produces a dependency on `io` which causes the final premise of T-UNSEAL to fail, because the type `string` of `io` is not wrapped by $\bullet_{\alpha}(-)$. For simplicity, we have so far dealt only with information escape through `io`. However, it turns out that we have introduced all the machinery we need to control information escape through general mutable references. We touch on this next.

4 Mutable References

Using the machinery already presented, we describe at a high level our strategy for handling mutable references. Writing to a reference can be modeled as an effect, so we represent it using the open modality. Each reference is associated to a dependency whose type is the type of the reference. Writing to the reference adds that dependency to the ambient set of dependencies. Consequently, writing to a reference inside the body of an `unseal` betrays having done so to the latter; the write can then be checked to not inappropriately leak data by checking that the type associated with reference's dependency (recall this is the same as the reference's type) places all its information under an appropriate quasi-closed modality; one might use the schema described in Section 3.3. That is, whereas reasoning about writing to references is a matter of the quasi-open modality, reasoning about reading is a matter of the quasi-closed modality.

Though we cannot make a finer point of it here, our approach to handling state in information flow—through the quasi-open and quasi-closed modalities—exceeds the expressiveness of prior designs. Prior approaches have instrumented the type system with a specialized *program counter* variable not treated as first-class by the language, and which thus cannot be abstracted over (by quantification) as both of our modalities can [2, 7, 11]. Accordingly, the scoping mechanics that give rise to the information borrowing pattern are not available there.

5 Related Work

Effect Systems. One might wonder whether we can simply enforce the absence of side effects in the presence of borrowed information, straightforwardly addressed by an effect system [6, 8, 9]. However, as mentioned, we care not just about whether computations perform effects, but further about *which* information those effects interact with. The ability to granularly discriminate on the data being revealed is essential. For instance, an effect system would not distinguish between `print("hi")`, where no information content is being revealed, and `print(pw)`, where the borrowed information in `pw` is being leaked. This lack of expressiveness does not appear to be resolvable through a conventional theory of effects alone. Our approach strikes a point in the design space providing both theoretical simplicity and highly concise specifications, minimally extending the theory of effects.

Borrowing Heap Cells. The notion of borrowing as a programming language feature is much more popularly recognized towards type systems for memory safety, rather than security typing. There, temporary access can be granted to access cells of memory or other resources, enabling reasoning about memory and concurrency safety [4, 12]. Though we have not formally developed the relationship between this technique and ours, we can give some intuition. Conventional borrowing is concerned exclusively with controlling the provenance of the contents of the heap, so need not pay mind to whether memory cells are passed to operations like `print`. On the other hand, information borrowing must concern itself with the contents of the *world*, because *information located anywhere* may still be relevant to it. As such, though seemingly sharing similar structure as suggested here, reasoning about information escape appears to require reasoning mechanisms with increased sensitivity to features like effects and complex control flow.

6 Conclusion

In this paper, we present information borrowing, which draws on the idea of borrowing from Rust to enable reasoning about non-escape. This provides a lightweight and highly readable specification mechanism for establishing security boundaries around unknown programs, granting assurance in passing them potentially sensitive data. Our method for doing so makes essential use of recent advances in information flow reasoning, additionally revealing an elegant approach to general mutable state in information flow.

Acknowledgments

This work was supported by the National Science Foundation under Grant No. CCF-2447499, the Department of Defense under Grant No. H98230-23-C-0275, and by seed funding

from the CyLab Security and Privacy Institute at Carnegie Mellon University.

References

- [1] Martín Abadi, Anindya Banerjee, Nevin Heintze, and Jon G. Riecke. 1999. A core calculus of dependency. In *Proceedings of the 26th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. ACM, San Antonio Texas USA, 147–160. doi:10.1145/292540.292555
- [2] Ethan Cecchetti, Andrew C. Myers, and Owen Arden. 2017. Non-malleable Information Flow Control. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*. Association for Computing Machinery, New York, NY, USA, 1875–1891. doi:10.1145/3133956.3134054
- [3] Hemant Gouni, Frank Pfenning, and Jonathan Aldrich. 2026. The Duality of Information Flow: Reconciling Robust Downgrading with Non-Interference. doi:10.48550/arXiv.2607.17445 arXiv:2607.17445 [cs.PL].
- [4] John Hogg. 1991. Islands: aliasing protection in object-oriented languages. In *Conference Proceedings on Object-Oriented Programming Systems, Languages, and Applications* (Phoenix, Arizona, USA) (OOPSLA '91). Association for Computing Machinery, New York, NY, USA, 271–285. doi:10.1145/117954.117975
- [5] Yiyun Liu, Jonathan Chan, Jessica Shi, and Stephanie Weirich. 2024. Internalizing Indistinguishability with Dependent Types. *Proceedings of the ACM on Programming Languages* 8, POPL (2024), 1298–1325. doi:10.1145/3632886
- [6] Eugenio Moggi et al. 1988. *Computational lambda-calculus and monads*. University of Edinburgh, Department of Computer Science, Laboratory for Foundations of Computer Science.
- [7] Andrew C. Myers. 1999. JFlow: practical mostly-static information flow control. In *Proceedings of the 26th ACM SIGPLAN-SIGACT symposium on Principles of programming languages (POPL '99)*. Association for Computing Machinery, New York, NY, USA, 228–241. doi:10.1145/292540.292561
- [8] Gordon Plotkin and John Power. 2003. Algebraic Operations and Generic Effects. *Applied Categorical Structures* 11, 1 (Feb. 2003), 69–94. doi:10.1023/A:1023064908962
- [9] Gordon D. Plotkin and Matija Pretnar. 2013. Handling Algebraic Effects. *Logical Methods in Computer Science* Volume 9, Issue 4 (Dec. 2013). doi:10.2168/LMCS-9(4:23)2013
- [10] Naokata Shikuma and Atsushi Igarashi. 2008. Proving Noninterference by a Fully Complete Translation to the Simply Typed lambda-calculus. *Logical Methods in Computer Science* Volume 4, Issue 3 (Sept. 2008). doi:10.2168/LMCS-4(3:10)2008
- [11] Dennis Volpano, Cynthia Irvine, and Geoffrey Smith. 1996. A sound type system for secure flow analysis. *Journal of Computer Security* 4, 2-3 (Jan. 1996), 167–187.
- [12] Ariel Weiss, Olek Gierczak, Daniel Patterson, and Amal Ahmed. 2021. Oxide: The Essence of Rust. doi:10.48550/arXiv.1903.00982 arXiv:1903.00982 [cs.PL].